Python Engineering - Reference Guide

CLASS 15 - Exceptions & Debugging | Syntax and tools

This is an extended reference: syntax, meaning, typical use, and common mistakes.

1. Exception

An exception is an object representing a runtime condition that interrupts the normal execution path.

```
float('12.5')  # OK
float('NOT_A_NUMBER')  # ValueError
```

The exception travels outward looking for a compatible handler.

2. try

Marks the code whose exceptions we intend to handle.

```
try:
    trajectory = load_trajectory_csv(file_path)
```

Keep try blocks focused; do not wrap half the program 'just in case'.

3. except ExceptionType

Handles a compatible exception raised inside try.

```
except FileNotFoundError:
    print('File does not exist')
```

4. as error

```
except ValueError as error:
    print(type(error).__name__)
    print(error)
```

The exception object gives access to its type and message.

5. else

Runs only when try completes without an exception.

```
try:
    data = load()
except ValueError:
    ...
else:
    analyze(data)
```

6. finally

Runs whether execution succeeded or failed.

```
try:
    ...
finally:
    print('Always runs')
```

For files, with open(...) already handles file closing, so finally is not required for that resource.

7. raise

Deliberately creates an exception.

```
if header != expected_header:
    raise ValueError('Invalid CSV header')
```

It turns an application rule into an explicit failure instead of allowing unreliable processing to continue.

8. ValueError

The operation is conceptually valid, but the supplied value is not acceptable.

```
float('abc')  # ValueError
```

9. FileNotFoundError

Raised when opening a path that does not exist.

```
open('missing.csv', 'r')  # FileNotFoundError
```

10. SyntaxError vs runtime exception

Type	When	Example
SyntaxError	Python cannot parse the code	if x > 0 (missing colon)
Runtime exception	Execution starts, then something fails	float('abc')

11. len()

```
len(['100.0','20.0','0.0'])  # 3
```

Used here to validate the expected column count.

12. enumerate()

```
for line_number, line in enumerate(file, start=2):
    ...
```

Provides a counter and item on each iteration; start changes the first counter value.

13. type(...).__name__

```
type(error).__name__  # 'ValueError'
```

A useful reminder that exceptions are objects with classes.

14. Avoid bare except

```
try:
    ...
except:
    print('Something failed')
```

It can hide failures we do not actually understand. Prefer specific exception types.

15. except Exception as error

Still broad. It can be appropriate at some application boundaries, but it is not the default for EX23.

```
except Exception as error:
    ...
```

16. Exception messages

```
# weak
raise ValueError('Error')

# better
raise ValueError(
    f'Line {line_number}: expected 3 columns, got {len(values)}'
)
```

17. Traceback

```
Traceback (most recent call last):
  File 'main.py', line 20, in main
    load_trajectory_csv(path)
  File 'file_io.py', line 31, in load_trajectory_csv
    v = float(values[1])
ValueError: could not convert string to float: 'NOT_A_NUMBER'
```

Practical reading: start at the bottom for type/message, then move upward to the relevant line in your code.

18. Breakpoint

Pauses execution before a line so current program state can be inspected.

```
v = float(values[1])  # set breakpoint before this
```

19. Step controls

Command	Simple meaning
Continue	Run until next breakpoint
Step Over	Execute current line without entering called functions
Step Into	Enter the called function
Step Out	Finish current function and return to caller

20. Variables and Call Stack

```
main()
  -> test_file()
      -> load_trajectory_csv()
          -> float(...)  # failure
```

Variables show current scope values; Call Stack shows how execution reached the current location.

21. Function contract

```
load_trajectory_csv(path)

expects:
- existing readable file
- expected header
- 3 numeric columns per row

returns:
- numerical trajectory array
```

Exceptions can communicate that a required part of that contract was violated.

22. Final cheat sheet

```
try:
    risky_operation()
except FileNotFoundError as error:
    handle_missing_file(error)
except ValueError as error:
    handle_bad_data(error)
else:
    use_successful_result()
finally:
    always_run_this()

raise ValueError('clear message')

DEBUG:
breakpoint -> inspect variables -> predict -> step

TRACEBACK:
bottom -> type/message -> move upward to your code
```